Description

This assessment consists of my implementation of a backpropagation algorithm in a multi-layer artificial neural network. Which are 2 layers for the network, that consists of 5 neurons. There are 3 inputs into neurons 4, 5 and 6, and the outputs of these neurons subsequently become the inputs for neurons 7 and 8 (Dreyfus, 1990). Having been provided a table of given inputs and target outputs within this assessment, a single epoch will require the algorithm to iteratively traverse through each of these 6 given inputs via the forward step process – whilst considering the effect of the network bias for every neuron (x0). Having reached the output layer of the network, the backpropagation algorithm will then be applied, which involves back-tracing to the start of the network, correct the weightings of the neuron connections which is calculated through the learning rate (which is provided as 0.1), and the error differentiation between each inputs target output and neuron 7 and 8's actual output for that input set – to optimise the performance of the algorithm (Chauvin and Rumelhart, 2013). For example, the first set of data to be included is [x1=0.50, x2=1.00, x3=0.75], the target output being Y1 which means for Y1 target output to equal 1, and Y2 target output to equal 0. Then for every input and target output set in the data, to consider the bias (x0) being an input of 1.

Learning Curve showing the Error Values as part of the Forward Steps

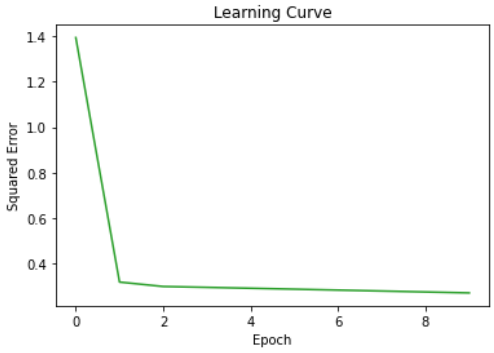


Figure 1 - Learning Curve for 10 Epochs

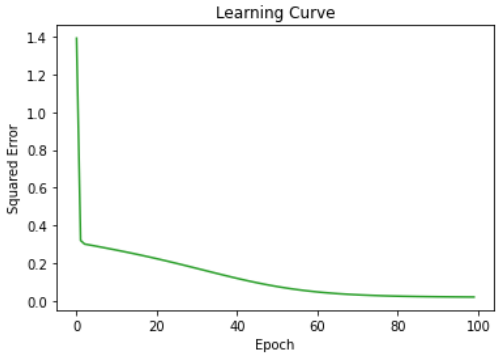


Figure 2 - Learning Curve for 100 Epochs

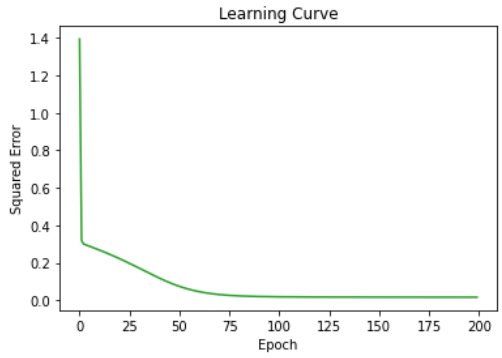


Figure 3 - Learning Curve for 200 Epochs

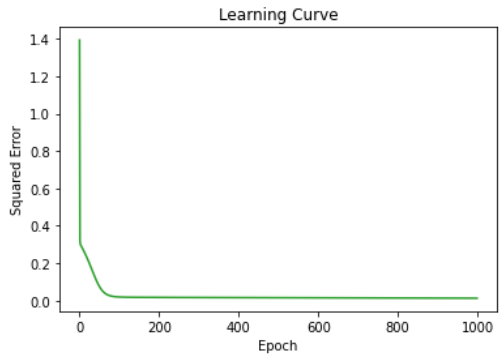


Figure 4 - Learning Curve for 1000 Epochs

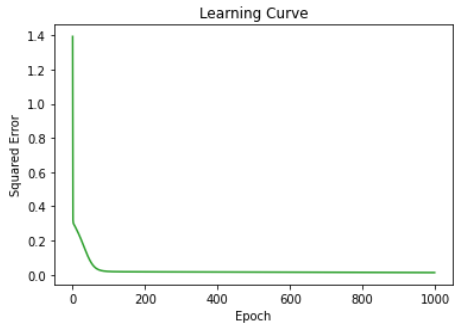
Table of Weights of the Backward Steps (up to 10 steps)

Step	0	1	2	3	4	5	6	7	8	9	10
w04	0.90	0.8727	0.8782	0.8840	0.8897	0.8951	0.9003	0.9052	0.9100	0.9145	0.9189
w14	0.74	0.7177	0.7152	0.7131	0.7111	0.7091	0.7072	0.7054	0.7036	0.7018	0.7001
w24	0.80	0.7715	0.7669	0.7628	0.7589	0.7550	0.7512	0.7476	0.7440	0.7405	0.7370
w34	0.35	0.3247	0.3214	0.3185	0.3157	0.3129	0.3102	0.3076	0.3051	0.3026	0.3002
w05	0.45	0.4162	0.4087	0.3988	0.3881	0.3770	0.3655	0.3537	0.3415	0.3289	0.3160
w15	0.13	0.1169	0.1219	0.1260	0.1300	0.1341	0.1384	0.1429	0.1475	0.1524	0.1574
w25	0.40	0.3811	0.3858	0.3896	0.3934	0.3974	0.4017	0.4061	0.4109	0.4158	0.4211
w35	0.97	0.9538	0.9585	0.9622	0.9659	0.9697	0.9738	0.9780	0.9824	0.9871	0.9919
w06	0.36	0.2447	0.2296	0.2229	0.2169	0.2106	0.2040	0.1971	0.1897	0.1820	0.1739
w16	0.68	0.6160	0.6105	0.6103	0.6111	0.6122	0.6137	0.6154	0.6174	0.6197	0.6222
w26	0.10	0.0230	0.0161	0.0153	0.0155	0.0163	0.0174	0.0189	0.0209	0.0231	0.0258
w36	0.96	0.8891	0.8829	0.8823	0.8827	0.8836	0.8848	0.8863	0.8882	0.8903	0.8927
w07	0.98	0.3700	0.2695	0.2377	0.2156	0.1950	0.1746	0.1545	0.1344	0.1144	0.0946
w47	0.35	-0.1618	-0.2258	-0.2299	-0.2259	-0.2209	-0.2160	-0.2113	-0.2067	-0.2024	-0.1983
w57	0.50	0.0421	-0.0098	-0.0073	0.0028	0.0142	0.0260	0.0380	0.0502	0.0625	0.0750
w67	0.90	0.4509	0.4037	0.4086	0.4207	0.4339	0.4471	0.4603	0.4736	0.4869	0.5002
w08	0.92	0.4773	0.4377	0.4521	0.4739	0.4965	0.5192	0.5417	0.5641	0.5864	0.6086
w48	0.80	0.3826	0.3212	0.3076	0.3007	0.2950	0.2896	0.2845	0.2796	0.2750	0.2705
w58	0.13	-0.2544	-0.3162	-0.3353	-0.3487	-0.3614	-0.3742	-0.3873	-0.4005	-0.4139	-0.4275
w68	0.80	0.4122	0.3486	0.3268	0.3108	0.2956	0.2805	0.2654	0.2504	0.2353	0.2202

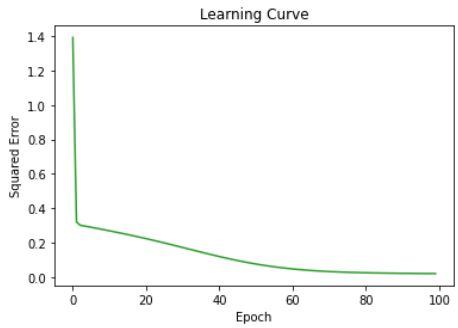
Explanation on when to stop training (how many training steps are needed)

To justify the termination condition when training a neural network, one method of approaching this is where to overcome overfitting, the data is split into testing and training data.

In training the data set for 1000 epochs, which is exceeding the required number of epochs to train greatly, obtains the following results when graphed:



With Epoch 100 being the rough point where overfitting beings, I then decide to avoid overfitting as much as possible since the accumulate square error difference post-100 epochs is in minimal difference as opposed to epochs 1 through 100. As shown in the graph below, where the stochastic curve decline approaches 0 for the squared error when the epochs approach 100:



Since the marginal distance of squared error is minimal post-100 epochs, it supports Tom M. Mitchell when he states, "*Given enough weight-tuning iterations, Backpropagation will often be able to create overly complex decision surfaces that fit noise in the training data or unrepresentative characteristics of the particular training sample*" (Mitchell, 1977).

Probability Distribution of Outputs Y as part of classifying the unseen input vector {x1=0.3, x2=0.7, x3=0.9}

For 10 Epochs

Neuron Output	Probability Distribution
Neuron 7 / y1	0.4270981637550399
Neuron 8 / y2	0.57290183624496

For 100 Epochs

Neuron Output	Probability Distribution
Neuron 7 / y1	0.6366295873254056
Neuron 8 / y2	0.36337041267459436

For 200 Epochs

Neuron Output	Probability Distribution

Neuron 7 / y1	0.6411680438941444
Neuron 8 / y2	0.35883195610585555

For 1000 Epochs

Neuron Output	Probability Distribution
Neuron 7 / y1	0.6500253871361228
Neuron 8 / y2	0.34997461286387727

Solution Code (*include text, no images)*

```
import math

import matplotlib.pyplot as plt


def plotLearningCurve(logged_error, graphname):

    x_data = []

    y_data = []

    x_data.extend([logged_error[i][0] for i in range(0, len(logged_error))])

    y_data.extend([logged_error[i][1] for i in range(0, len(logged_error))])

    fig, ax = plt.subplots()

    plt.title(graphname)

    ax.set(xlabel='Epoch', ylabel='Squared Error')

    ax.plot(x_data, y_data, 'tab:green')

    plt.show()


def calc_net (inputs, weights):

    net = 0

    for i in range(len(inputs)):

        net += inputs[i] * weights[i]

    return net


def function_calculation(net):

    return (1 / (1 + ((math.e)**(-1*net))))


inputs_and_outputs = [[1,0.5,1.00,0.75,1,0],[1,1.00,0.50,0.75,1,0],[1,1.00,1.00,1.00,1,0],[1,-
0.01,0.50,0.25,0,1],[1,0.50,-0.25,0.13,0,1],[1,0.01,0.02,0.05,0,1]]
```

```
neuron_4_weights = [0.90, 0.74, 0.8, 0.35] #w04, w14, w24, w34
neuron_5_weights = [0.45, 0.13, 0.40, 0.97] #w05, w15, w25, w35
neuron_6_weights = [0.36, 0.68, 0.10, 0.96] #w06, w16, w26, w36
neuron_7_weights = [0.98, 0.35, 0.50, 0.90] #w07, w47, w57, w67
neuron_8_weights = [0.92, 0.80, 0.13, 0.80] #w08, w48, w58, w68
neuron_7_output = 0
neuron_8_output = 0

logged_error = []

def Backpropagation(neuron_456_inputs, steps):
    current_lowest_epoch = 0
    current_lowest_epoch_error = 10
    for i in range(steps):
        sum_of_epoch_squared_errors = 0
        print("\n\n::::::::::::::::::::::::::::::::: STEP ", i+1, "::::::::::::::::::::::::::::::::::::::")
        for j in neuron_456_inputs:
            current_inputs = j[0:4] # x0, x1, x2, x3
            print("\n\nneurons_456_inputs::: ", current_inputs)
            neuron_7_desired_output = j[4]
            print("neuron_7_desired_output::: ", neuron_7_desired_output)
            neuron_8_desired_output = j[5]
            print("neuron_8_desired_output::: ", neuron_8_desired_output)

            #Forward Step
            neuron_4_output = function_calculation(calc_net(current_inputs, neuron_4_weights))
            neuron_5_output = function_calculation(calc_net(current_inputs, neuron_5_weights))
            neuron_6_output = function_calculation(calc_net(current_inputs, neuron_6_weights))

            neuron_78_inputs = [j[0], neuron_4_output, neuron_5_output, neuron_6_output] # j[0] ==
bias
            neuron_7_output = calc_net(neuron_78_inputs, neuron_7_weights)
            neuron_8_output = calc_net(neuron_78_inputs, neuron_8_weights)
```

```
#Backward Step
#Output Errors
output_7_error = neuron_7_desired_output - neuron_7_output
output_8_error = neuron_8_desired_output - neuron_8_output

#Hidden Errors
hidden_4_error = neuron_4_output * (1 - neuron_4_output) * ((neuron_7_weights[1] * output_7_error)+(neuron_8_weights[1] * output_8_error))
hidden_5_error = neuron_5_output * (1 - neuron_5_output) * ((neuron_7_weights[2] * output_7_error)+(neuron_8_weights[2] * output_8_error))
hidden_6_error = neuron_6_output * (1 - neuron_6_output) * ((neuron_7_weights[3] * output_7_error)+(neuron_8_weights[3] * output_8_error))

#weight updates
learning_rate = 0.1
update04 = learning_rate * hidden_4_error * current_inputs[0]
update14 = learning_rate * hidden_4_error * current_inputs[1]
update24 = learning_rate * hidden_4_error * current_inputs[2]
update34 = learning_rate * hidden_4_error * current_inputs[3]
update05 = learning_rate * hidden_5_error * current_inputs[0]
update15 = learning_rate * hidden_5_error * current_inputs[1]
update25 = learning_rate * hidden_5_error * current_inputs[2]
update35 = learning_rate * hidden_5_error * current_inputs[3]
update06 = learning_rate * hidden_6_error * current_inputs[0]
update16 = learning_rate * hidden_6_error * current_inputs[1]
update26 = learning_rate * hidden_6_error * current_inputs[2]
update36 = learning_rate * hidden_6_error * current_inputs[3]

update07 = learning_rate * output_7_error * current_inputs[0]
update47 = learning_rate * output_7_error * neuron_4_output
update57 = learning_rate * output_7_error * neuron_5_output
update67 = learning_rate * output_7_error * neuron_6_output
update08 = learning_rate * output_8_error * current_inputs[0]
```

```
update48 = learning_rate * output_8_error * neuron_4_output
update58 = learning_rate * output_8_error * neuron_5_output
update68 = learning_rate * output_8_error * neuron_6_output

#update changes
neuron_4_weights[0] += update04
print("W04: ", neuron_4_weights[0])
neuron_4_weights[1] += update14
print("W14: ", neuron_4_weights[1])
neuron_4_weights[2] += update24
print("W24: ", neuron_4_weights[2])
neuron_4_weights[3] += update34
print("W34: ", neuron_4_weights[3])
neuron_5_weights[0] += update05
print("W05: ", neuron_5_weights[0])
neuron_5_weights[1] += update15
print("W15: ", neuron_5_weights[1])
neuron_5_weights[2] += update25
print("W25: ", neuron_5_weights[2])
neuron_5_weights[3] += update35
print("W35: ", neuron_5_weights[3])
neuron_6_weights[0] += update06
print("W06: ", neuron_6_weights[0])
neuron_6_weights[1] += update16
print("W16: ", neuron_6_weights[1])
neuron_6_weights[2] += update26
print("W26: ", neuron_6_weights[2])
neuron_6_weights[3] += update36
print("W36: ", neuron_6_weights[3])
neuron_7_weights[0] += update07
print("W07: ", neuron_7_weights[0])
neuron_7_weights[1] += update47
```

```
        print("W47: ", neuron_7_weights[1])
        neuron_7_weights[2] += update57
        print("W57: ", neuron_7_weights[2])
        neuron_7_weights[3] += update67
        print("W67: ", neuron_7_weights[3])
        neuron_8_weights[0] += update08
        print("W08: ", neuron_8_weights[0])
        neuron_8_weights[1] += update48
        print("W48: ", neuron_8_weights[1])
        neuron_8_weights[2] += update58
        print("W58: ", neuron_8_weights[2])
        neuron_8_weights[3] += update68
        print("W68: ", neuron_8_weights[3])

        sum_of_epoch_squared_errors += ((output_7_error)**2) + ((output_8_error)**2)

        #Softmax Function
        print("\nProbability Distribution Neuron 7::: ", ((math.e)**(neuron_7_output)) /
(((math.e)**(neuron_7_output)) + ((math.e)**(neuron_8_output))))
        print("Probability Distribution Neuron 8::: ", ((math.e)**(neuron_8_output)) /
(((math.e)**(neuron_7_output)) + ((math.e)**(neuron_8_output))))

    logged_error.append([i, ((sum_of_epoch_squared_errors)/12)])
    if sum_of_epoch_squared_errors/12 < current_lowest_epoch_error:
        current_lowest_epoch_error = sum_of_epoch_squared_errors/12
        current_lowest_epoch = i+1

  print("current_lowest_epoch:: ", current_lowest_epoch)
  print("current_lowest_epoch_error:: ", current_lowest_epoch_error)

Backpropagation(inputs_and_outputs, 10) #Training wih 1000 epochs, Testing with 100 epochs
plotLearningCurve(logged_error, "Learning Curve")
Backpropagation([[1, 0.3, 0.7, 0.9, 1, 0]], 1)
```

References

Chauvin, Y. and Rumelhart, D.E., 2013. *Backpropagation: theory, architectures, and applications. Chapter 1.* Available from *[*https://www.taylorfrancis.com/books/mono/10.4324/9780203763247/backpropagation-yves-chauvin-david-rumelhart*]*

Dreyfus, S.E., 1990. *Artificial neural networks, back propagation, and the Kelly-Bryson gradient procedureI.* Vol. 13, No.5, pp. 926-928. Available from [https://arc.aiaa.org/doi/abs/10.2514/3.25422?journalCode=jgcd]

Mitchell, T.M., 1997. *Machine Learning.* pp. 81-126